

§ CRUD complets des ressources §

PRÉREQUIS

Avant de pouvoir commencer la séance, vous devez :

1. avoir fini la séance 3.1 ;

OBJECTIFS DE LA SÉANCE

1. Comprendre et appliquer les principes du CRUD.
2. Développer des routes REST pour un serveur Express en relation avec la donnée **Patient**.
3. Pouvoir écrire un fichier http.
4. Créer une route GET avec paramètre.
5. Créer des routes POST et PUT pour créer et mettre à jour un patient.
6. Utiliser un fichier http pour tester les routes.

RETOUR EN ARRIÈRE

Dans nos dernières séances, nous avons créé des routes permettant de récupérer (lire, READ) et créer (CREATE) des données de patients et docteurs. Nous avons également mis en place des mappers et des DTO. Ces mappers et DTO nous accompagneront tout au long de notre projet.

EXERCICE 1: ROUTE POUR MODIFIER UN DOCTEUR

Préparez le fichier `http/doctors.http` pour ajouter une requête PUT qui permettra de modifier un docteur. La requête PUT envoie ses données dans le **corps de la requête**. Le type de données reçue par le contrôleur est `DoctorDTO` (qui contient tous les champs d'un docteur).

Créez une nouvelle requête dans votre fichier `doctors.http`. Créez une route dans votre contrôleur `doctors.controller.ts`, affichez un message de log et validez avec votre fichier http que la route fonctionne.

Créez une fonction du mapper `DoctorsMapper` pour convertir le docteur passé dans le corps de la requête (`DoctorDTO`) en `Doctor`.

Ensuite, implémentez le *happy path* pour cette route :

1. vérifiez que les informations envoyées dans le corps de la requête sont bien de type `Doctor` à l'aide de la fonction guard `isDoctor`, renvoyez une erreur `400` si les informations ne sont pas correctes ;
2. vérifiez que l'id passé en paramètre est bien présent et est un nombre, renvoyez une erreur `400` si ce n'est pas le cas ;
3. vérifiez que l'id passé dans le corps de la requête correspond à celui passé en paramètre, envoyez une erreur `400` si ce n'est pas le cas ;
4. vérifiez que le docteur existe dans la liste des docteurs, renvoyez une erreur `404` si le docteur n'existe pas ;
5. remplacez dans la liste le docteur trouvé par le docteur passé dans le corps de la requête ;
6. renvoyez le docteur modifié avec un code de retour `200`.

EXERCICE 2: ROUTE POUR SUPPRIMER UN DOCTEUR

Préparez le fichier `http/doctors.http` pour ajouter une requête DELETE qui permettra de supprimer un docteur.

Dans le contrôleur, ajoutez une requête DELETE permettant de supprimer un docteur sur base de l'ID passé en paramètre directement dans l'URL.

Le *happy path* pour cette route est :

1. vérifiez que l'ID passé en paramètre est bien un nombre, renvoyez une erreur `400` si l'ID n'est pas un nombre ;
2. vérifiez que le docteur existe dans la liste des docteurs, renvoyez une erreur `404` si le docteur n'existe pas ;
3. supprimez le docteur de la liste des docteurs à l'aide de la méthode `splice` ;
4. renvoyez un code de retour `200` si le docteur a été supprimé.

EXERCICE 3: FILTRER LA LISTE DES DOCTEURS

Au sein du fichier `doctor.model.ts` créez une interface `DoctorFilter` qui contiendra le champ optionnel `speciality`.

Préparez le fichier `http/doctors.http` pour ajouter une requête GET `/doctors` qui permettra d'obtenir la liste filtrée des docteurs. Le paramètre de la requête sera `speciality`. Nous obtenons ainsi la liste des docteurs qui correspondent à la spécialité passée en paramètre. Attention que ce paramètre est optionnel.

Reprenez le contrôleur et modifiez la route GET `/doctors` pour qu'elle permette de filtrer la liste des docteurs. Le paramètre de la requête sera `speciality`. Nous obtenons ainsi la liste des docteurs qui correspondent à la spécialité passée en paramètre.

EXERCICE 4: CRUD DU PATIENT

Créez les différentes requêtes dans le fichier `http/patients.http` ainsi que les routes dans le fichier `controllers/patients.controller.ts` pour les opérations CRUD suivantes :

1. Créez une route POST pour créer un patient :
 - Cette route prend en entrée un `NewPatientDTO`. Attention, le champ `birthdate` est de type `string`.
 - Le `NewPatientDTO` est converti en `NewPatient` à l'aide de la fonction `fromNewDTO` du `PatientsMapper`. Au sein de cet object, le champ `birthdate` est converti en `Date`. La fonction pour créer une date est `new Date(dto.birthdate)`.
 - a
2. Créez une route PUT pour mettre à jour un patient.
3. Créez une route DELETE pour supprimer un patient.

Il est important de toujours **commencer** par créer un jeu de données test. Cela nous donne confiance dans notre code tout au long du développement. **Tester son code de manière régulière** est une bonne pratique de développement.

Il serait tentant de copier/coller du code et de légèrement adapter les noms de variables. Cependant, il est important de bien comprendre chaque étape et de les implémenter soi-même. Cela n'est en rien une perte de temps, bien au contraire. C'est en faisant, par soi-même, de nombreuses fois, la même chose que l'on apprend le mieux.

EXERCICE 5: CONNEXION AVEC LE FRONT-END

Maintenant que notre serveur est capable de créer, mettre à jour et supprimer des données, nous allons connecter notre front-end à notre serveur. Dans le monde de l'entreprise, le front-end et le back-end sont souvent développés par des équipes différentes. Le développement et les tests associés sont distincts. La mise en commun des deux parties (serveur et client) s'effectue parfois en fin de projet. Il est donc important de savoir développer son serveur de manière indépendante du front-end.

Récupérez sur Moodle le fichier `patient-html.zip` et décompressez-le. Placez son contenu dans un dossier `src/html/patient`.

Le fichier HTML est composé de trois formulaires : le premier permet de demander un patient en fonction de son identifiant, le second permet de mettre à jour ce patient et le dernier permet de créer un nouveau patient.

Utilisez les différents formulaires pour vérifier le fonctionnement de vos routes. Le code HTML et CSS est extrêmement simple et basic, n'hésitez pas à les modifier pour rendre l'interface plus agréable. Le code JS n'est pas à modifier ni à comprendre pour le moment.

CONCLUSION

Cette semaine, nous avons progressé dans l'implémentation du CRUD. Nous avons vu comment créer, mettre à jour et supprimer des données. Nous avons vu comment tester nos routes à l'aide du fichier `http`.

Enfin, nous avons connecté notre serveur à un front-end simple pour tester nos routes dans le cadre d'une application complète.